

数据预处理与 特征工程

主讲：高刚毅

邮箱：ggy0222@sina.com

部门：信智学院 人工智能系 9-411

Content

01 | 数据获取

02 | 数据预处理

03 | 特征工程

04 | 阶段考核要求

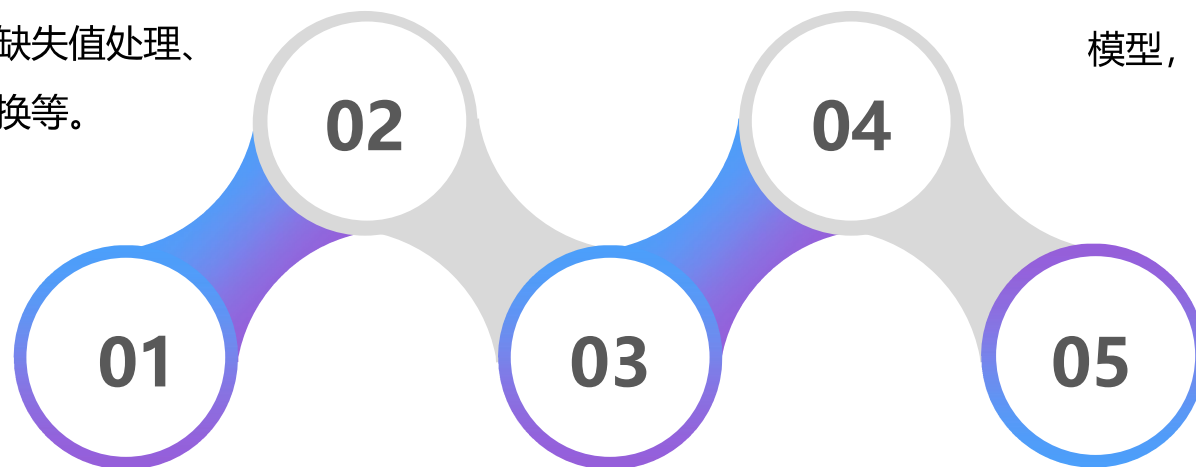
数据处理及模型训练一般流程

2 数据预处理

原始数据往往不能满足要求，需要做一些预处理，例如：缺失值处理、归一化、编码、数据变换等。

4 模型选择与评估

综合对比候选模型，选择最适合模型，并进行参数调优。



1 获取数据

首先获取原始数据，然后从业务需求的角度出发，理解数据及其含义。

3 特征工程

从原始数据中，提取、构造、优化特征等。

5 模型预测

部署模型，应用阶段对新样本进行预测。

数据获取

- 数据获取途径与来源:

1. 内部业务系统（如数据库、日志文件等）
2. 外部公开数据或第三方接口数据
3. 设备采集数据
4. 网络爬取数据等

- 主要风险:

1. 数据异构性：不同来源的数据格式差异大
2. 质量问题：存在缺失值、重复记录或噪声数据
3. 隐私与合规风险：可能涉及隐私或违法平台协议



原始数据存在的主要问题

数据不属于
同一量纲



信息冗余



定性特征不能
直接使用



缺失值或异
常值



连续型特征

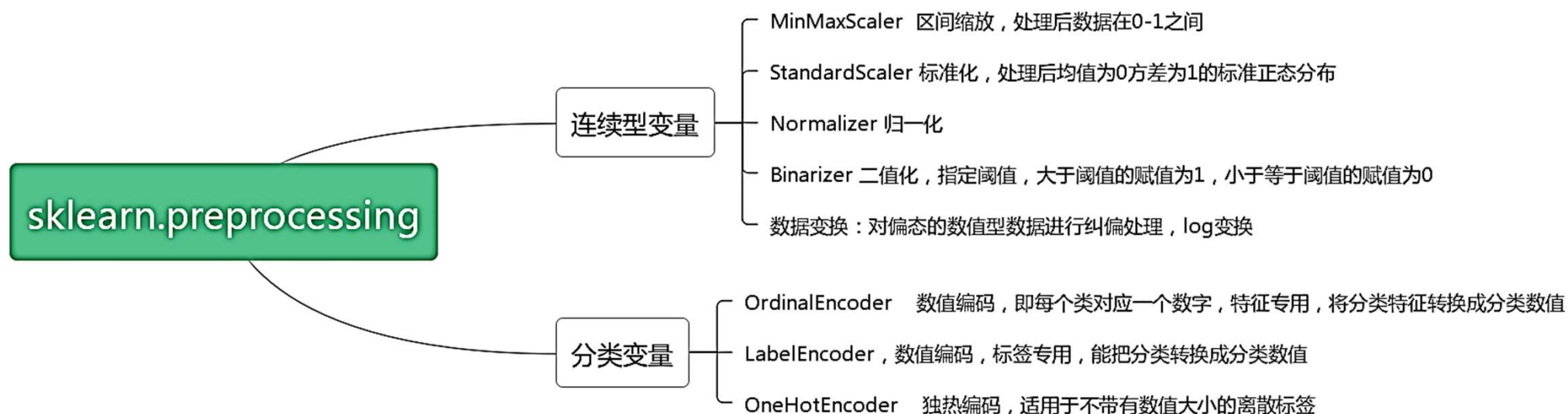


数据预处理

- **数据预处理**：是指在进行**数据分析或建模之前**，对原始数据进行**清洗、整合、转换和编码**等一系列操作，以提升数据质量与可用性。
- **数据预处理的主要作用**：它是数据科学流程中的**关键基础步骤**，直接影响后续**分析结果的准确性与模型性能**。
- **数据预处理主要内容**：
 1. 缺失值处理
 2. 连续型特征的无量纲化
 3. 特征编码
 4. 连续型特征离散化
 5. 数据变换（调整数据分布）
 6. 异常值处理（离群值）

sklearn 预处理的相关函数

- **sklearn** (全称**scikit-learn**) 是一个基于 Python 的开源机器学习库, 提供了从数据预处理到模型评估的一整套工具。
- sklearn 提供了一些常用的数据预处理功能:
- <https://scikit-learn.org/1.4/modules/classes.html#module-sklearn.preprocessing>



数据预处理

1、缺失值处理：

- **缺失值**是指数据集中某些字段**观测值未被记录或无法获取**的情况，通常用NaN、NULL、空字符串或占位符（如-999）表示。
- 根据缺失比例、数据分布和业务场景选择合适方法：**直接删除**；**填充法**（均值、中值、众数、固定值、前后向填充、插值法等）；**保留缺失**（作为特殊类别）
- `sklearn.impute.SimpleImputer` (`missing_values=nan`, `strategy=' mean'` ,`fill_value=None`, `copy=True`)

参数	含义&输入
<code>missing_values</code>	告诉SimpleImputer，数据中的缺失值长什么样，默认空值np.nan
<code>strategy</code>	我们填补缺失值的策略，默认均值。 输入"mean"使用均值填补（仅对数值型特征可用） 输入"median"用中值填补（仅对数值型特征可用） 输入"most_frequent"用众数填补（对数值型和字符型特征都可用） 输入"constant"表示请参考参数"fill_value"中的值（对数值型和字符型特征都可用）
<code>fill_value</code>	当参数strategy为"constant"的时候可用，可输入字符串或数字表示要填充的值，常用0
<code>copy</code>	默认为True，将创建特征矩阵的副本，反之则会将缺失值填补到原本的特征矩阵中去。

数据预处理

2、连续型特征的无量纲化：

- **无量纲化**：是指通过数学变换，将不同特征的数值范围调整到**同一量级**，消除因单位、量纲或数值范围差异导致的模型偏差。
- **无量纲化目的**：让不同特征在模型训练中具有**可比性**，避免量纲大的特征主导计算。
 - ① **模型公平性**：例如，身高（单位：米，范围1.5~2.0）和体重（单位：千克，范围50~100）数值差异巨大，若不处理，模型会过度依赖体重特征。
 - ② **优化算法稳定性**：梯度下降等算法在量纲差异大的数据上**收敛速度慢**。
 - ③ **距离计算合理性**：KNN、SVM等依赖距离的模型，量纲差异会导致**距离计算失真**。
- 无量纲化可以是线性的，也可以是非线性的。
- **线性的无量纲化**主要包括：**标准化**、**区间缩放**、**归一化**。

数据预处理

• 线性无量纲化的常用方法：

① 标准化：(preprocessing.StandardScaler)

- 将数据转换为均值为0、标准差为1的正态分布，即标准正态分布。
- 在PCA，聚类，支持向量机，神经网络等算法中，标准化往往是最好的选择。
- 示例：原始数据：[50, 60, 70, 80, 90]，标准化后：[-1.41, -0.71, 0, 0.71, 1.41]

$$X_{\text{new}} = \frac{x - \mu}{\sigma}$$

② 区间缩放：(preprocessing.MinMaxScaler)

- 将数据缩放到一个特定的范围，通常是[0,1]。
- 缺点：对异常值非常敏感。
- 示例：原始数据：[50, 60, 70, 80, 90]，归一化后：[0, 0.25, 0.5, 0.75, 1]

$$X_{\text{new}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

③ 归一化：(preprocessing.Normalizer)

- 将每个样本的向量归一化为单位范数（如L1、L2）。
- 示例：原始样本行：[3, 4]
- L2归一化后：[3/5, 4/5] = [0.6, 0.8]（使得样本向量的模长为1）

- L1范数：每个样本除以该样本的绝对值之和

$$x_{\text{new}} = \frac{x}{\sum |x_i|}$$

- L2范数（默认）：每个样本除以该样本的欧氏距离

$$x_{\text{new}} = \frac{x}{\sqrt{\sum x_i^2}}$$

数据预处理

3、特征编码：

- 特征编码是将原始数据中的非数值型特征（如类别、文本）转换为数值型表示的过程，以便机器学习模型能够处理和学习。
 - ① 标签编码（`preprocessing.LabelEncoder`）：为每个类别标签分配一个唯一整数编号
 - ② 序号编码（`preprocessing.OrdinalEncoder`）：通常用于处理具有大小关系的特征数据，如产品等级分为高、中、低三档，转换后依次为3、2、1，保持大小关系
 - ③ 独热编码（`preprocessing.OneHotEncoder`）：将特征变成0和1组成的二进制值，适合无序分类变量；当特征取值很多时，会导致维度爆炸

Color	
Red	
Red	
Yellow	
Green	
Yellow	



Red	Yellow	Green
1	0	0
1	0	0
0	1	0
0	0	1
0	1	0

数据预处理：离散特征编码方法

数据预处理

4、连续型特征离散化：

- 包括：二值化（0或1）、分箱（分区间）

① 二值化：通常指基于阈值将连续特征的值映射到 0 和 1（或-1和1）。

- `class sklearn.preprocessing.Binarizer(*, threshold=0.0, copy=True)`
 - `threshold`：阈值，大于阈值设置为1，小于阈值设置为0，默认 0.0
 - `copy`：默认为True，设置为False将就地二进制定化并避免复制

② 分箱：将数值型数据划分为多个区间进行编码，如年龄分成“青年”“中年”“老年”。

- `class sklearn.preprocessing.KBinsDiscretizer(n_bins=5, *, encode='onehot', strategy='quantile', dtype=None, subsample=200000, random_state=None)`
 - `n_bins`：分箱的数量（默认5）。`encode`：编码方式（'onehot'，'onehot-dense'，'ordinal'，默认 'onehot'）。`strategy`：分箱策略（'uniform' 等宽分箱，'quantile' 等频分箱，'kmeans' 聚类分箱，默认 'quantile'）。

数据预处理

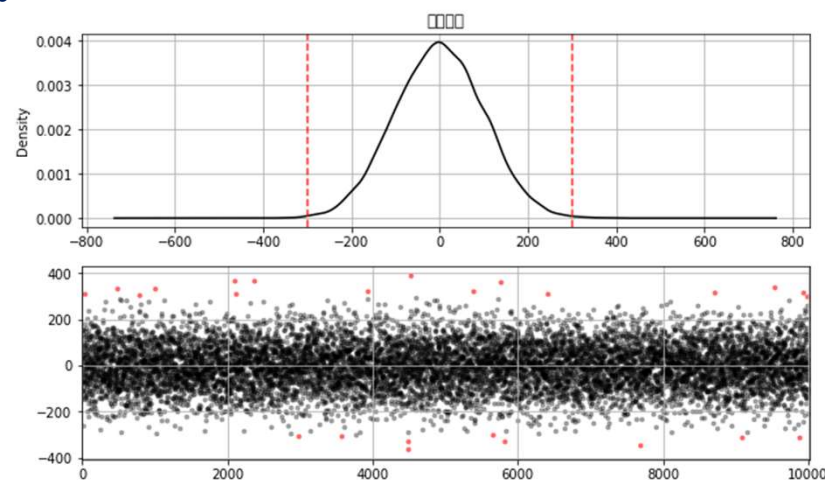
5、数据变换：

- **数据变换**：指的是对原始数据进行系统性的转换，目的是改善数据分布、适应算法需求、消除偏差等，从而提高数据的质量和适用性。
- 可通过直方图 (hist) 来查看特征字段的数据分布情况。
- **数据偏态分析**：可使用 scipy 或 Pandas 的 **skew** 函数
 - **偏态系数**以平均值与中位数之差对标准差之比率来衡量偏斜的程度，用SK表示偏斜系数。
 - **负偏**：偏态系数小于0时，平均数在众数之左，是一种左偏的分布。
 - **正偏**：偏态系数大于0时，均值在众数之右，是一种右偏的分布。
 - **0偏度**：偏态系数等于0，数据分布对称，接近正态分布。
- **数据纠偏方法**：**对数变换**，即**Log变换**，对数据进行对数处理，常用于处理偏态分布的数据，使其更接近正态分布。（可使用 `np.log10()` 函数）

数据预处理

6、异常值处理：

- 异常值 (Outliers) 是数据集中显著偏离其他观测值的极端值，也称离群点。
- 异常值产生的原因：
 - 数据采集错误：传感器故障、人工录入错误。
 - 自然变异：罕见事件或极端情况（如超高收入人群）。
 - 数据处理错误：单位换算错误或数据拼接问题。
- 异常值分析方法：
 - 3σ 原则：即大于 3σ 的数据为异常值。
 - 可视化检测：绘制箱线图或散点图分析。
- 异常值处理方法：
 - 删除异常数据；用统计值替换异常值；对数变换等。



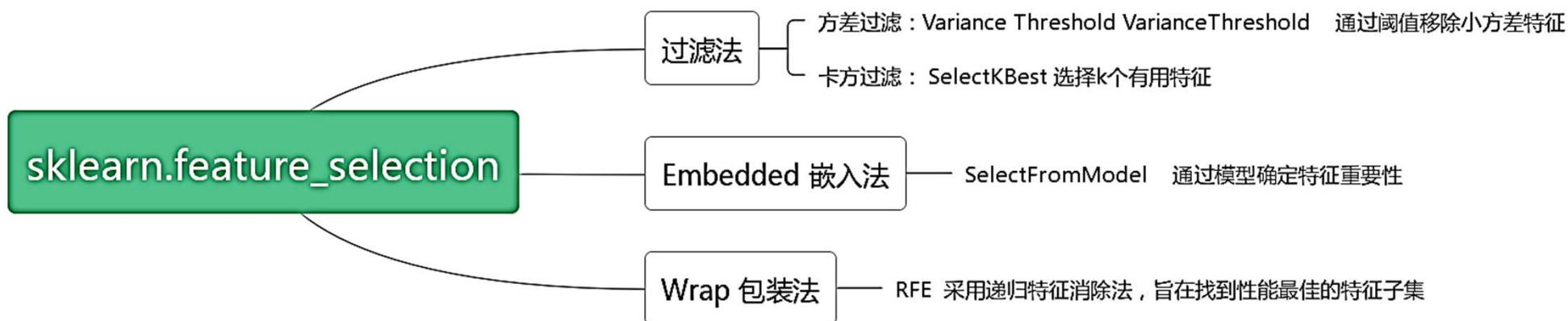
特征工程

- **特征工程**：是利用领域知识从原始数据中**提取特征**（特性、属性）的过程。
- 机器学习中有句经典的台词是：**数据和特征决定学习上限，模型和算法只是逼近这个上限**，可见特征工程在机器学习中的重要性。
- 传统机器学习中的特征工程主要包括两方面：
 1. 特征维度**过多**时，为加快模型训练速度，进行特征选择即**过滤**掉不重要的特征；
 2. 特征**较少**或模型训练性能不好时，可通过对问题的理解尝试**构建特征**提升维度。
- **特征工程的目的**：1) 降低计算成本，2) 提升模型上限。

特征提取 (feature extraction)	特征创造 (feature creation)	特征选择 (feature selection)
从文字，图像，声音等其他非结构化数据中提取新信息作为特征。比如说，从淘宝宝贝的名称中提取出产品类别，产品颜色，是否是网红产品等等。	把现有特征进行组合，或互相计算，得到新的特征。比如说，我们有一列特征是速度，一列特征是距离，我们就可以通过让两列相处，创造新的特征：通过距离所花的时间。	从所有的特征中，选择出有意义，对模型有帮助的特征，以避免必须将所有特征都导入模型去训练的情况。 CSDN @Grateful_Dead424

sklearn 库的相关函数

- 当数据预处理完成后，我们需要**选择有意义的特征**输入机器学习的算法和模型进行训练。通常来说，从两个方面考虑来选择特征：
 - 特征是否发散**：如果一个特征不发散，例如方差接近于0，也就是说样本在这个特征上基本上没有差异，这个特征对于样本的区分并没有什么用。
 - 特征与目标的相关性**：与目标相关性高的特征，应当优选选择。
- 首先应尽量**理解业务**，另外，**可参考以下几种方法**：



特征选择方法

- 2.1 过滤法：
- 方差过滤：（移除低方差的特征）
 - 函数： `class sklearn.feature_selection.VarianceThreshold(threshold=0.0)`
 - **threshold**: float, 阈值, 训练集方差低于此阈值的要素将被删除。默认设置是使所有要素的方差均非零, 即删除所有样本中具有相同值的特征。
- 阈值很小, 被过滤掉得特征比较少, 对模型不会有太大影响;
- 阈值比较大, 被过滤掉的特征有很多, 模型可能变好也可能变差。
- 如果事先进行了归一化, 那方差法就没有意义了。

特征选择方法

- 2.1 过滤法：
- 卡方过滤：（相关性过滤）
 - 卡方过滤是专门针对离散型标签（即分类问题）的相关性过滤。
 - 函数：sklearn.feature_selection.chi2(X, y)
 - X 是输入的样本特征矩阵，必须为非负值；y 目标标签向量（类别标签）。
 - 卡方检验类 feature_selection.chi2 计算每个非负特征和标签之间的卡方统计量，并依照卡方统计量由高到低为特征排名。
 - 还可以结合 feature_selection.SelectKBest 可以输入“评分标准”来选出前K个分数最高的特征的类，可以借此除去与分类目的无关的特征。
 - sklearn.feature_selection.SelectKBest(chi2, k=10)

特征选择方法

- 2.2 Embedded 嵌入法:

- 函数: `class sklearn.feature_selection.SelectFromModel` (estimator, threshold=None, prefit=False, norm_order=1, max_features=None)
- 说明: `SelectFromModel` 是一个通用转换器, 可以与任何在拟合后具有 `coef_`, `feature_importances_` 属性或参数中可选惩罚项的评估器一起使用, 常见的适用模型包括线性回归、逻辑回归、Lasso、随机森林等。
- estimator (object): 构建转换器的基础模型; threshold: 用于特征选择的阈值。
- 对于有 `feature_importances_` 的模型来说, 若重要性低于提供的阈值参数, 则认为这些特征不重要并被移除。`feature_importances_` 的取值范围是[0,1], 如果设置阈值很小, 比如 0.001, 就可以删除那些对标签预测完全没贡献的特征。如果设置得很接近1, 可能只有一两个特征能够被留下。

特征选择方法

- 2.3 包装法： (Wrapper)

- 它是一种特征选择方法，其核心思想是将特征选择过程与特定的机器学习模型训练过程紧密结合。它通过评估不同特征子集对模型性能（如准确率、召回率等）的影响，来筛选出最优的特征组合。
- 函数： `class sklearn.feature_selection.RFE (estimator, n_features_to_select=None, step=1, verbose=0)`
 - 参数 **estimator** 是需要填写的实例化后的评估器， **n_features_to_select** 是想要选择的特征个数， **step** 表示每次迭代中希望移除的特征个数。
 - 包装法是一个特征选择和算法训练同时进行的方法，使用一个目标函数作为黑盒来选取特征，而不是输入某个评估指标或统计量的阈值。
 - 区别于过滤法和嵌入法的一次训练解决所有问题，包装法要使用特征子集进行多次训练，因此它所需要的计算成本是最高的。

特征选择方法

- 几种特征选择方法比较：

1. 过滤法，完全独立于模型，更快速，成本低，但更粗糙。
2. 包装法，紧密依赖特定模型，计算成本高，需多次训练模型。
3. 嵌入法，在模型训练过程中自动完成，计算成本中等，随模型训练一并完成。

- 过滤法不依赖模型，可作为特征工程的第一道筛选，选择结果可能不够精准，属于“粗筛”。包装法适用于特征数量适中、追求极致性能、有充足计算资源的场景。嵌入法适用于中等维度数据、需要平衡效率与性能的场景，如金融风控、推荐系统。

如何选择合适的特征选择方法？

1. **先用过滤法做初步筛选**：使用方差过滤、相关性分析等快速去除明显无用特征，降低后续计算负担。
2. **根据模型类型选择后续策略**：
 - 若使用线性模型（如逻辑回归、Lasso），优先考虑嵌入法；
 - 若使用树模型（如随机森林、XGBoost），可直接利用其`feature_importances_`进行嵌入式选择；
 - 若追求最高精度且资源充足，可使用包装法（如RFE）进一步优化。
3. **结合业务理解人工干预**
 - 所有自动方法都应辅以领域知识判断，避免误删关键业务特征。
4. **推荐组合使用流程**：
 - 过滤法 → 嵌入法 → 包装法（可选）
 - 先粗筛，再精挑，最后微调，实现效率与性能的平衡。

特征工程：降维

- **降维：**指的是降低特征矩阵中特征的数量。
- **降维的目的：**
 - 让算法运算更快，效果更好
 - 利用降维进行数据可视化，帮助理解数据的分布。
- **Sklearn中降维算法：**
 - sklearn.decomposition 矩阵分解

sklearn.decomposition 矩阵分解

这个sklearn.decomposition模块包括矩阵分解算法，包括PCA、NMF或ICA等。该模块的大部分算法都可以看作是一种降维技术。

用户指南：见分量中的信号分解(矩阵因式分解问题)一节以了解更多细节。

decomposition.DictionaryLearning([...])	字典学习
decomposition.FactorAnalysis([n_分量, ...])	因子分析(FA)
decomposition.FastICA([n_分量, ...])	FastICA：一种独立分量分析的快速算法。
decomposition.IncrementalPCA([n_分量, ...])	增量主成分分析(IPCA)。
decomposition.KernelPCA([n_分量, ...])	核主元分析(KPCA)。
decomposition.LatentDirichletAllocation([...])	基于在线变分贝叶斯算法的潜Dirichlet分配
decomposition.MinibatchDictionaryLearning([...])	小批量词典学习
decomposition.MinibatchSparsePCA([...])	小批量稀疏主成分分析
decomposition.NMF([n_Components, init, ...])	非负矩阵分解(NMF)
decomposition.PCA([n_Components, Copy, ...])	主成分分析(PCA)
decomposition.SparsePCA([n_分量, ...])	稀疏主成分分析(SparsePCA)
decomposition.SparseCoder(字典, *, [...])	稀疏编码
decomposition.TruncatedSVD([n_分量, ...])	用截断SVD(又名LSA)进行降维。

decomposition.dict_learning(X, n分量, ...)	解决了一个字典学习矩阵分解问题。
decomposition.dict_learning_online(X[, ...])	在线求解字典学习矩阵分解问题。
decomposition.fastica(X[, n_分量, ...])	执行快速独立分量分析。
decomposition.non_negative_factorization(X)	计算非负矩阵分解(NMF)。
decomposition.sparse_encode(X, 字典, *)	稀疏编码

<https://www.kaggle.com/robertmiller/sklearn-decomposition>

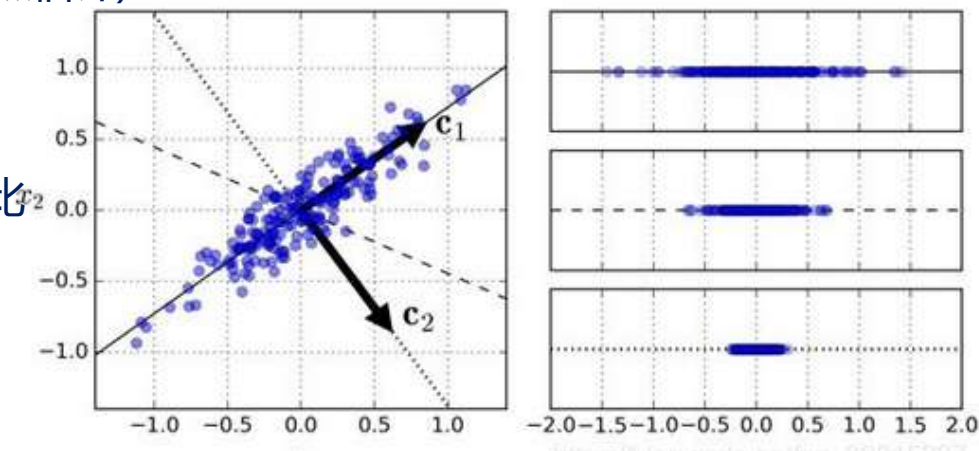
特征工程：降维

- **主成分分析法 (Principal Component Analysis)**：最流行的降维算法。首先它找到接近数据集分布的超平面，然后将所有的数据都投影到这个超平面上。
- `class sklearn.decomposition.PCA (n_components=None, copy=True, whiten=False, svd_solver=' auto' , tol=0.0, iterated_power=' auto' , random_state=None)`

主要参数： `n_components`：维度，整数或“mle”（最大似然估计）
`svd_solver`：默认‘ auto’ 。运行不同的求解器。

常用属性： `explained_variance_`：可解释性方差
`explained_variance_ratio_`：可解释性方差百分比
`n_components_`：估计的特征数量。

接口： `fit`
`transform`
`fit_transform`
`inverse_transform` 转换回原维度



阶段考核要求

- **阶段目标：**理解数据预处理和特征工程的概念和主要内容；掌握使用 sklearn 库进行数据预处理和特征选择的主要方法和技术。
- 1. **数据预处理：**
 - 首先，读取数据文件，并进行数据查看和分析；
 - 其次，完成缺失值、连续型特征无量纲化处理、特征编码、连续特征离散化、数据变换、异常值处理等数据预处理任务。
- 2. **特征工程：**
 - 首先，分析理解每个特征字段的含义，进行特征的初步人工筛选；
 - 综合运用过滤法、嵌入法、包装法等进行特征选择，确定最终保留的字段。
- **阶段考核：**建议采用 Jupyter Notebook 分步展示数据预处理和特征工程的过程。

The image features a dark blue background with a complex, glowing circuit board pattern. In the center, there is a dark blue, rounded cloud-like shape. Inside this shape, the Chinese text "谢谢观看!" (Thank you for watching!) is written in a bold, white, sans-serif font. Several bright blue lines, resembling circuit traces or data streams, enter the cloud from the top and exit from the bottom, creating a sense of connectivity and flow.

谢谢观看!